Senior Design Autobot Project — Navilott: Part One

Mike Orduna
Ana Carlos
Ignacio Rubio

October 11, 2026

Contents

1 Abstract **1**

2 Overview **1**
2.1 Purpose of This Report 1
2.2 The Team 1
2.3 Development Process 1
2.4 How to Read This Report 1

3 Problem **1**
3.1 Objective 1
3.2 Constraints 1
3.3 The Course 2
3.4 Requirements 2

I Design **3**

4 Architecture **3**
4.1 Structure of an Autonomous Robot 3
4.2 Single-Board Design 4
4.3 Additional Hardware 5
4.4 Software Language 7
4.5 Background Threads 8
4.6 Linux Scheduler 10
4.7 Tooling 10
4.8 Software Architecture 11

5 Capture **11**
5.1 Camera Path 11
5.2 Design Decisions 11
5.3 Failure Handling 11
5.4 Open Questions 12

6 Perception **12**
6.1 Approach 12
6.2 Stage Order 12
6.3 Lane Boundaries and Offset 12
6.4 Stop Lines 12
6.5 Traffic Lights 12
6.6 Stop Signs 12
6.7 Debugging Perception 12

7 Estimation **13**
7.1 Role . . . 13
7.2 Lane Filter . . . 13
7.3 Heading Tracker . . . 13
7.4 Voting . . . 13
7.5 The Packet . . . 13

8 Navigation **13**
8.1 Scope and Attribution . . . 13
8.2 The Contract . . . 13
8.3 Driving Rules . . . 13
8.4 Measured Robot Response . . . 14
8.5 Intersections and Route . . . 14

II Evidence 15

9 Testing **15**
9.1 Scope . . . 15
9.2 Testing Without the Robot . . . 15
9.3 The Suite . . . 15
9.4 What Testing Proves . . . 15

10 Diagnostics **15**
10.1 Why Instrument the Pi . . . 15
10.2 What Is Recorded . . . 15
10.3 Observer Effect . . . 16
10.4 What It Showed . . . 16

11 Verification **16**
11.1 What Counts as Verified . . . 16
11.2 Routines . . . 16
11.3 Checks by Requirement . . . 16
11.4 Whole-Robot Checks . . . 16

12 Results **16**
12.1 Status by Requirement . . . 16
12.2 Routine Results . . . 17
12.3 Course Runs . . . 17
12.4 Measured Robot Response . . . 17

III Assessment 18

13 Limitations **18**
13.1 Requirements Not Met 18
13.2 Requirements Not Yet Shown 18
13.3 Departures from the PRD 18
13.4 Limits of the Design 18

14 Discussion **18**
14.1 What Worked 18
14.2 What Didn't 18
14.3 What I Would Do Differently 18
14.4 What Remains 18

15 Conclusion **19**

IV Supplements **20**

A Hardware **20**

B Parameters **20**
B.1 Capture 20
B.2 Perception 20
B.3 Estimation 20

C The Estimation Packet **20**

D Repository **21**
D.1 Layout 21
D.2 Commands 21

1 Abstract

[TODO: Abstract.]

2 Overview

2.1 Purpose of This Report

[TODO: Why write a self-evaluation at all? Who is the reader (you, an interviewer, the team)? What period of work does it cover, and as of what date and commit?]

2.2 The Team

[TODO: Who owned which subsystem? What did each contribute to the parts this report covers?]

2.3 Development Process

[TODO: How was the work actually done: what did you design and write yourself, what came from AI-assisted sessions, and how did you review and verify that work?]

2.4 How to Read This Report

[TODO: A short map of the parts. Write this last.]

3 Problem

3.1 Objective

[TODO: What must the robot do, in one paragraph? Which PRD rules matter most to vision: right-lane driving, the exact traffic-light rules (green to enter, yellow only once inside, never on red), the stop sign at the exit?] [TODO: Obstacle avoidance: The PRD's executive summary states it as a must. Was it formally downgraded? By whom, and when?]

3.2 Constraints

[TODO: Which constraints actually shaped your design decisions, and which were just boxes to tick? Consider: $100 BOM, the PCB requirement, 12 V limit, size (10 x 11 x 11 cm), local processing only, start button and 5 s countdown, elapsed time in tenths of a second.] [TODO: Battery display: the PRD asks the power system to alert navigation. Why was battery life moved to the display?]

3.3 The Course

[TODO: Describe the course the robot was built and tested on. Where does it differ from the PRD's course, and does each difference make the vision problem easier or harder?]

Table 1: **[TODO: caption]**

PRD	Team course
[TODO: rows]	

3.4 Requirements

[TODO: Turn the PRD's prose into requirements that can each be checked one way. For each: is it straight from the PRD, or derived (something vision needs so a PRD rule can be met)? What numbers will you hold yourself to, and why those numbers?] [TODO: The PRD's validation guidance asks for each object's detection field (range, angles, minimum size). Its own requirement, or part of the detection ones?]

Table 2: **[TODO: caption]**

ID	Requirement	Source
[TODO: rows]		

Part I
Design

4 Architecture

4.1 Structure of an Autonomous Robot

An autonomous robot has two main goals: complete its task without human intervention and perform it efficiently. Efficiency as a metric depends entirely on what we are designing for — an autonomous robot that can navigate a track on its own. For the case of Navilott, this means a robot that can navigate a street-style course on its own — following a lane, stopping at stop signs and obeying traffic lights.

A robot's architecture. In robotics, this is often called "autonomous navigation", a part of "autonomous control," where the robot controls itself to perform a task, such as following a line or avoiding obstacles (Douglas, 2020b). A robot can have many different methods to measure and sense its environment — camera, sonar, ultrasonic, IR, etc. Subprocesses in the robot's software can use these measurements to form a map of the environment. The robot's estimation system can use the map to calculate the robot's position and orientation in the world. The robot's navigation system can then use these estimates to determine the robot's decision about what to do next. The robot's control system can then actuate motors in response to these decisions. Actuation is performed in two layers: the control loop which sends PWM signals, and the driver which switches battery voltages based on the PWM signals that actuate the motors via current. Feedback from encoders on the motor can then be routed back into estimation and navigation systems to ensure that the decision was taken in the right place.

The robot can then be divided into 6 main areas:

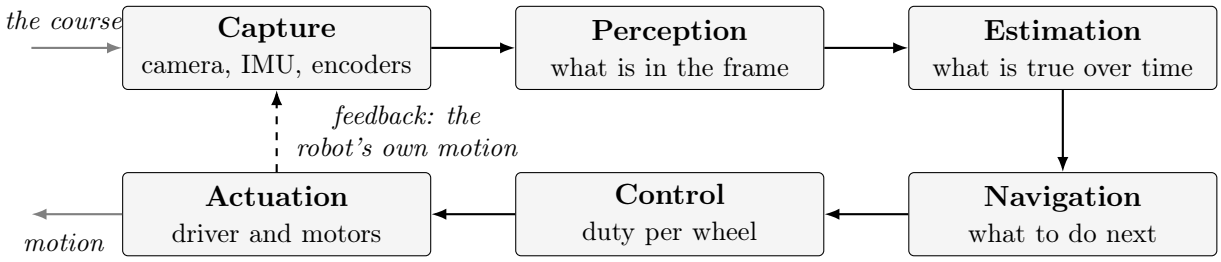


Figure 1: The six areas as a closed loop.

- **Capture**: the robot's sensors and how they capture the environment
- **Perception**: the robot's processing algorithms and how they interpret those measurements
- **Estimation**: the robot's estimation system and how it generalizes those measurements

- **Navigation**: the robot's navigation system and how it forms decisions about the environment based on those perceptions
- **Control**: the robot's control system and how it moves the robot around those decisions
- **Actuation**: the robot's actuation system and how it actually moves the robot

4.2 Single-Board Design

Our first concern was how the start of the stack would interact with the end of it — meaning how we would connect the robot's perception system to the motor actuation. To achieve this, three options were considered:

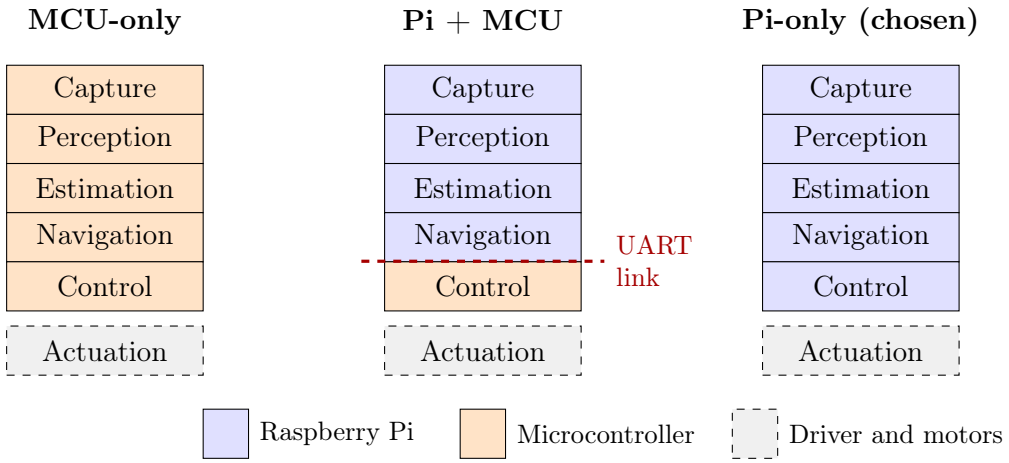


Figure 2: Where each area runs under the three options.

- **MCU-only**: An STM32 or Arduino runs the perception pipeline and drives the motors directly.
- **Pi-only**: The Pi runs the perception pipeline and drives the motors directly via PWM signals to a motor driver.
- **Pi + MCU**: The Pi processes perception and sends a packet over UART to an STM32 or Arduino, which executes a fast, real-time PID loop and generates the PWM signals.

Why not MCU-only? The Raspberry Pi was a necessary addition because of our robot's camera-centric design. Intense computer vision needs a lot of RAM to run efficiently, ISP and CSI-2 interface to capture video quickly and bypass CPU overhead, and OpenCV library to perform the intense image processing we would require. For instance, to process a frame that is 480×270 in color pixels would require 389 KB of RAM — more than what is found in most microcontrollers.

How a Pi + MCU would affect architecture. Integrating an MCU into our design would require a new protocol to transfer the data between the Pi and the MCU, synchronization of clocks between the two devices (the Pi captures and stamps frames based on its clock while encoders from the motors would work on the MCU's clock), and managing two separate codebases. These different challenges, while making the architecture more true to a real-world robot, would also make it more difficult to maintain.

Decision: Pi-only. As such, a single-board design was chosen (The Raspberry Pi Zero 2 W) — which sacrifices speed of a dedicated motor control loop for the simplicity and testability of the Pi, as well as accepting the overhead from the Linux scheduler. However, these tradeoffs were considered when developing the stack and are discussed in Section 4.6.

4.3 Additional Hardware

Mapping the baseline architecture for the design introduced several new avenues to explore for integrating hardware components. For a robotic system, a camera alone would not be enough to function as a robot. How does the robot stay powered, how does it move, what else would it use to sense its environment, how can we monitor its internal state, and how does the control system communicate with the user? Taking in these questions, we decided to add the following hardware components:

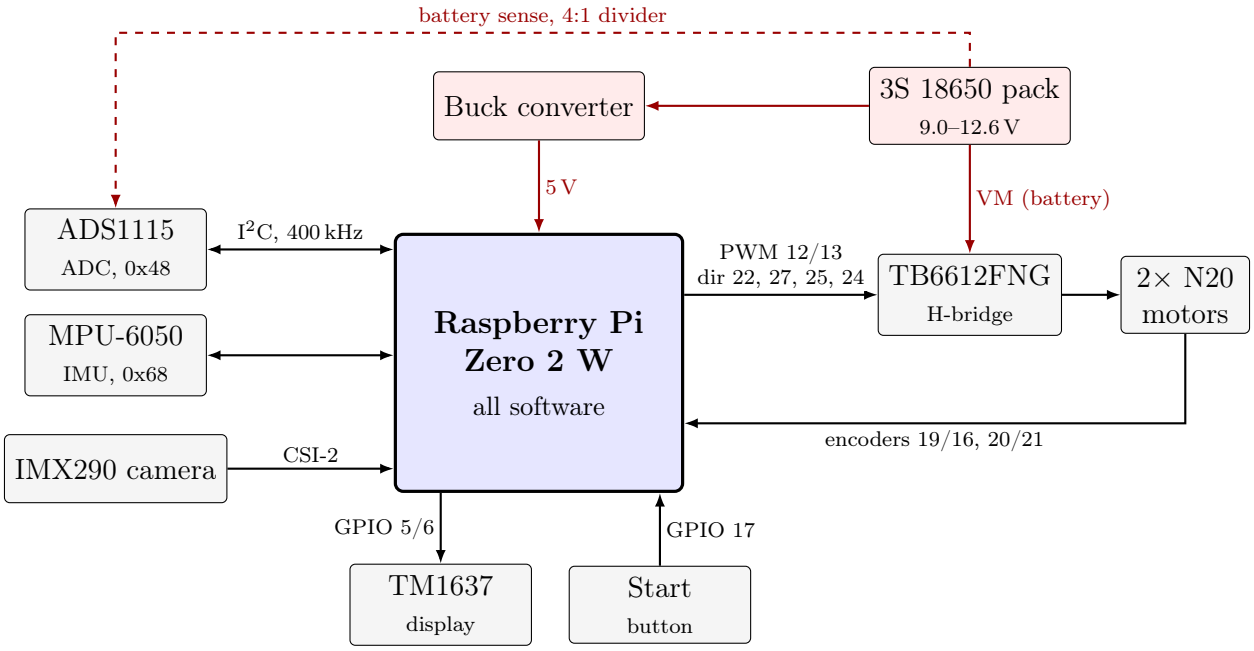


Figure 3: Hardware added around the Pi.

Table 3: Hardware added around the Pi.

Part	Role	Reason	Area
Innomaker's Cam-MIPI290RAW	Main vision sensor for the robot.	Allows the robot to visually see its environment.	Capture
ADS1115	Measures the battery voltage; Pi has no analog input.	Allows for our team to report the battery status to the user.	Capture (Battery)
MPU6050	Measures the robot's angular rate and acceleration.	Allows us to measure the robots yaw and integrate it into our estimation system.	Capture → Estimation
TB6612FNG	The robot's motor driver; Pi GPIO can't supply motor current.	Allows for precise control of the robot's motors.	Actuation
2 N20 Motors	The robot's actuation system.	Allows for the robot to move.	Actuation
Encoders	Measures the robot's wheel speeds.	Allows us to estimate the robot's actual speed.	Estimation
TM1637	The robot's run-time interface.	Allows us to display information about the robot's state to the user.	Interface
Start Button	The robot's start button.	Allows us to physically start the robot.	Interface
DC-DC Buck Converter	The Pi's battery converter.	Allows a steady 5V power supply.	Power
3S 18650 Battery Cells	The robot's battery pack.	Allows us to power the robot's hardware.	Power

Camera. Originally, we were using a Raspberry Pi Cam V2 module as our camera, which featured an IMX219 sensor. However, its field of view is cropped by the mode selection of the Pi. This added a heavy blindspot of 22 cm to the robot's vision and made it incapable of viewing both the lane lines and the traffic light in one frame while at a stop line. Details can be found in Section 5. In response, we opted for another type of camera: the Innomaker's Cam-MIPI290RAW, a wide-angle camera that is compatible with the Raspberry Pi's CSI-2 interface. This allowed us to achieve a fuller field of view and reduced our blindspot down to 10 cm, but at the cost of the lens producing a distortion effect which would require software to correct (Section 5).

Battery monitoring. Intrinsically, the Pi does not have an analog input and thus does not have a way to capture any voltage running through it — it requires some external component to monitor it — the batteries feature no Battery Management System (BMS). That is where the ADS1115 comes into play. As per Section 3, we are to warn the system

after falling beneath 10.5 V and stop navigation after the robot falls under 9.9 V.

IMU. Our robot will need a way to measure and verify its angular turns during intersection logic and lane keeping. The Pi has no internal gyroscope that can measure its angular velocity, nor its acceleration. The MPU6050 fills this gap, supplying the Estimation system with additional information about the robot's yaw. Details can be found in Section 7.

Actuation. The robot will need some way to actually move around the course as well as a way to accurately control how current is sent to the motors. The Pi's power pins can only supply a few milliamps for other peripherals connected to it. However, these pins cannot supply a current large enough to move the motors, lest the Pi becomes damaged. In this, the TB6612FNG motor driver switches battery power to the N20 motors via an H-Bridge. Details can be found in Section 4.8.

Encoder. To measure the robot's speed, we need a way to measure the speed of the wheels. The Pi has no internal encoder that can measure the speed of the wheels. The Encoders are used to measure the speed of the wheels. Details can be found in Section 7.

Interface. Finally, a way to communicate with the user about the robots state, time limits, and a way to physically start the robot are required (Section 3). In this, the TM1637 7-segment display and a small button are used. The display is used to show navigation information and the time while the robot runs through a course (and to display its final time) while the button doesn't allow for the motor to actually process software commands until it is pressed. Details can be found in Section 4.8.

Power. To power on all of these different peripherals, the robot uses 3S 18650 battery cells, 12.6 V full, 11.1 V nominal, 9.0 V empty. The buck converter is used to normalize and distribute 5V power rails — it is used to power the Pi while the batteries are directly routed to the motor driver. The Pi powers the remaining components via its 3.3V pin.

PCB. The PRD requires the integration of the main SBC and its hardware components to be mounted via a printed circuit board (PCB). All of the hardware components (except the camera) are routed through the PCB, with the chassis serving as mounting points for motors, the camera, the display, the button, and the batteries. The PCB uses headers that are soldered to the PCB to connect to each component — each component has a place on the board it sits and cables that route power and data to any of the externally seated components listed earlier.

4.4 Software Language

Due to the complexity of the problem, we required a language that had access to complex computer vision algorithms, data visualization, and iteration speed. Because of these needs, we chose Python because it featured 3 main solutions to our problem:

- **GStreamer**: Python's bindings for OpenCV allows it to access GStreamer from its backend, which is an open-source multimedia framework. It provides access to the camera, and allowed us to interface with the camera and process the frame data from it.
- **OpenCV**: Python's bindings for OpenCV, a computer vision library, provided access to the complex algorithms that we needed to perform our computer vision tasks.
- **Testing**: Python allows us to abstract away the complexities of the different algorithms and libraries that we needed to use. Because of this, we were able to take advantage of different techniques to test, such as duck-typing and mocking, to verify that our system performed as expected.

GStreamer. For interfacing with the camera, Python had access to the GStreamer library, an open-source multimedia framework written in C. It works on multiple threads, chaining together elements — data flows from a source element, through various filter/transform elements, and finally ends at a sink element.

OpenCV. For computer vision, Python has access to the OpenCV library, an open-source computer vision library written in C++. It contains a wide range of algorithms for image processing, object recognition, and more.

Global interpreter lock. However, accepting Python as our language was not without its tradeoffs. Python runs in an interpreter, and because its types are only checked at runtime, each operation costs far more than the equivalent in C. Python also has a Global Interpreter Lock (GIL), a lock that the interpreter holds while it runs Python code. It prevents multiple threads of Python bytecode to run in parallel across different cores — only a single thread can run Python bytecode at a time, on whichever core Linux gives it.

Python as an acceptable tradeoff. Although Python's cost per operation is higher than C's, Python's overhead is acceptable because the per-pixel work never runs in Python — it is handled by C/C++ code.

4.5 Background Threads

The codebase splits up its processes across 5 different kinds of threads:

- **Main Loop**: This is the main thread that runs the main loop of the program. It handles the 4-phase pipeline and the communication between the main loop and the other threads.
- **SensorHub**: This thread is responsible for reading from the IMU and the encoders every 10 ms, loading into a buffer until it's drained. It remains asleep for most of the robot's life.

- **Motor Watchdog**: This thread is responsible for braking the motors if it does not receive a command from the main pipeline for 0.5 seconds. But if Linux kills the Python process, the watchdog dies with it, while pigpio keeps the PWM at its last duty.
- **Pigpio Callbacks**: This thread reads and delivers encoder edges to the total encoder counter.
- **GStreamer/libcamera**: GStreamer backend interfaces with the Pi's libcamera software to feed the main loop (OpenCV's VideoCapture) with the camera's video stream.

Pigpio. The pigpio daemon is its own process that runs in the background of the Pi. It is initialized during the system's startup and remains running while it is on. It timestamps the encoder edges and owns the pins controlling the motors, keeping their PWM signals online — even if the main loop stalls.

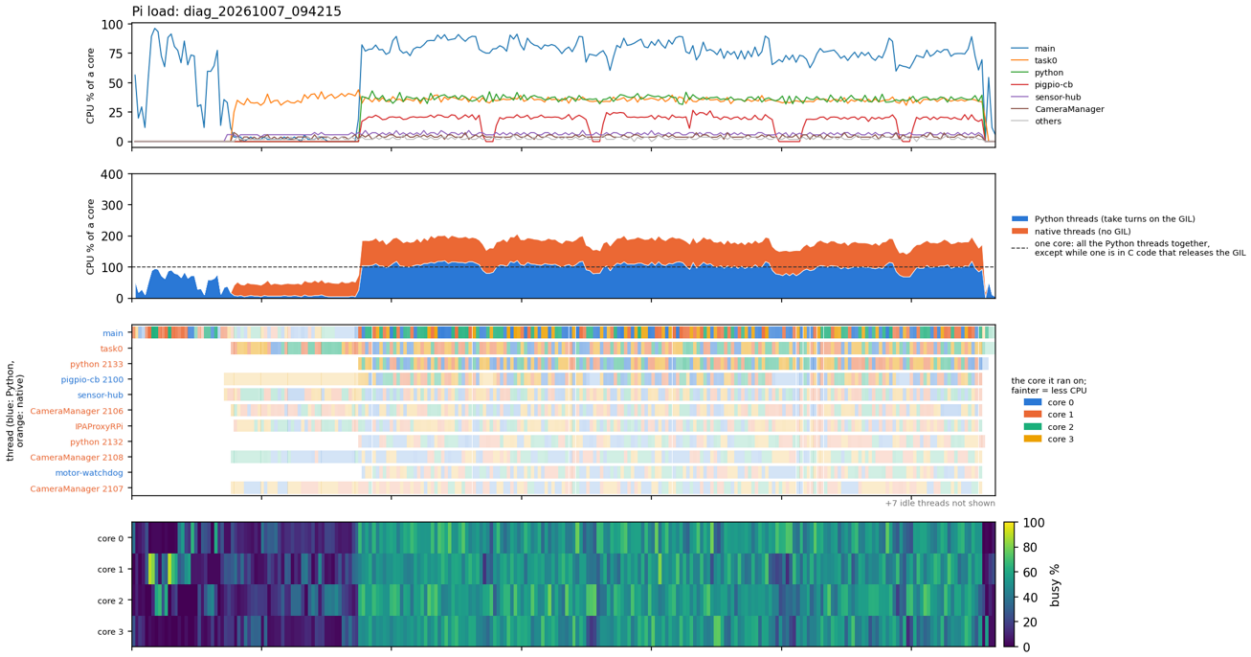


Figure 4: Pi load during a live run with the diagnostics recorder in a separate process (2026-10-07). Top to bottom: CPU per thread; Python threads against native threads; the core each thread ran on; per-core load; temperature and clock; memory.

How the GIL splits work with OpenCV. When the program starts, the main loop holds the GIL. Whenever it calls into OpenCV it releases the lock, and a waiting thread — the sensor hub, the motor watchdog or the pigpio callbacks — can take it. While the main loop runs Python code instead, a waiting thread must wait: only after 5 ms does Python ask the holder to release the lock, unless the thread lets it go.

How the sensor hub works with the GIL. For the sensor hub, which reads the IMU every 10 ms, a wait of up to 5 ms is a large share of its period. Two changes shorten how long each reading needs the lock. Each reading fetches the IMU's 14-byte sample block in a single I^2C transfer and then releases the GIL once it's done. The I^2C bus runs at 400 kHz so each transfer finishes sooner. Every reading is also timestamped when it is taken, so a late read is recorded at its true time and does not become a heading error.

Where does GStreamer fit? The GStreamer pipeline runs a C thread and is separate from the GIL, allowing it to perform its own tasks without interrupting the main loop. However, it spends a lot of its time waiting for the next frame.

4.6 Linux Scheduler

4.7 Tooling

[TODO: What tools were used to develop the software? What was gained and lost by using these tools? How did the tools affect the architecture?]

The tooling of the Pi was actually an afterthought when it came to developing the software — how to reproduce the same environment across multiple Pis and how to manage daily friction. As problems arose, we designed tooling to address them.

Consistency and reproducibility. During the early stages of development, many of the components were wired up on a breadboard. We had a few issues with power management which caused some components to pull more current than expected, damaging them — the Pi being one of them. Instead of manually reinstalling each dependency, possibly missing some dependencies in the environment, we decided to use a Makefile script to install them — which would exclude the ones that we would not keep in the final design. This was a direct countermeasure to avoid cluttering the Pi with failed and dropped packages and to make the Pi more reproducible. This was the purpose of our first tool, the `setup.mk` file.

Daily friction. However, additional initialization was needed on every boot: the clock on the Pi always started unsynchronized and was a major source of frustration — preventing updates to the system and causing naming conflicts. The school's network would block the Pi's connection to the NTP server and since the Pi has no battery-backed clock, it could not keep time while the Pi was powered off. Furthermore, the Pi utilized a virtual environment to run the robot's Python code. On every boot, we had to set the clock by hand, activate the environment, and launch pigpio. To make the process easier, we implemented a script which did all three. This was our second tool, the `session.mk` file.

Python virtual environment. The virtual environment we used was set up to mitigate OS conflicts between installed libraries and packages:

- Many of our drivers — Adafruit and TM1637 libraries — come from pip. We often ran into issues in the terminal installing them because the OS would refuse their installation to protect its Python environment.

- The robot packages could conflict with some of the OS packages.
- A broken package could be easily removed and reinstalled.
- We also needed to expose the virtual environment to packages installed with `apt` — such as Picamera2 — which binds to the system's libcamera.

The virtual environment sees the foundation of the camera stack and holds the robot's own pip packages.

Making an installable package. With the virtual environment initialized, the code needed to be set up as an installable package. A lack of organization across the codebase made it difficult to run and test after each iterative change. Python projects often describe themselves in a `pyproject.toml` file, which describes the project's metadata and dependencies. After calling `pip install -e .`, the project is now an installable package — it installs the package as a link to the source folder so any changes are reflected and don't need to be reinstalled. `setup.mk` runs this command automatically.

4.8 Software Architecture

[TODO: After deciding on the main architecture and hardware peripherals to use, how was the software architecture designed? What were some of the key decisions made and why? How did the software evolve over time and what changes, if any, were made to the architecture?]

5 Capture

5.1 Camera Path

[TODO: How does light become a frame in Python? Which stages does the frame pass through, and which of them did you configure?]

5.2 Design Decisions

[TODO: Which capture decisions mattered, and what would have gone wrong without each? Consider the pinned sensor mode, the output size, keeping only the newest frame, flipping in GStreamer, and who assigns frame ids and timestamps.]

5.3 Failure Handling

[TODO: What happens when a frame is dropped, and when the camera dies? Which requirement does this serve, and how do you know it works?]

5.4 Open Questions

[TODO: What about capture is still unmeasured or uncertain? For example, the delay from exposure to timestamp, or exposure settings under course lighting.]

6 Perception

6.1 Approach

[TODO: Why classical computer vision rather than a learned model? What about this course made that a reasonable bet, and what did you give up?]

6.2 Stage Order

[TODO: What are the stages, in order, and why that order? Why is there no memory between frames at this phase?]

6.3 Lane Boundaries and Offset

[TODO: How are lane lines found, and how does a set of lines become one steering error? What does the sign of the offset mean? What happens when only one line is visible?]

6.4 Stop Lines

[TODO: How is a stop line told apart from a lane line? How is its distance measured, and in what units?]

6.5 Traffic Lights

[TODO: What made traffic lights hard on this course? How did the approach change once you found the lights were bare LEDs (glow mode)?]

6.6 Stop Signs

[TODO: How are stop signs detected, and how well tuned is the detector on real course frames?]

6.7 Debugging Perception

[TODO: How do you see why a frame failed? How do you see why a gate failed? Why would it be important to see this at all?]

7 Estimation

7.1 Role

[TODO: Where is the line between perception and estimation, and why draw it there?]

7.2 Lane Filter

[TODO: How is a raw offset smoothed, and when is a frame not trusted? What do vision, hold and stale mean, and why those thresholds?]

7.3 Heading Tracker

[TODO: What does the IMU add while vision is lost? What does heading actually measure, and what are its limits?]

7.4 Voting

[TODO: Why vote across frames for traffic lights, stop signs and stop lines? Why a window of three?]

7.5 The Packet

[TODO: What does navigation receive each frame? Which fields matter most? (Full field list in Appendix C.)]

8 Navigation

8.1 Scope and Attribution

[TODO: What is the role of navigation, and how was the design chosen? What was considered and what were the tradeoffs?]

8.2 The Contract

[TODO: What is the interface between vision and navigation? What must a navigator always do, and what must it never do? Why fix this before navigation existed?]

8.3 Driving Rules

[TODO: How does the robot respond to each light state, stop signs, stop lines and a lost lane? Do these rules match the PRD?]

8.4 Measured Robot Response

[TODO: How the robot actually responds to commands? How does that compare to the contract?]

8.5 Intersections and Route

[TODO: How is a turn sequenced, and how does the robot know which way to go at each intersection?]

Part II
Evidence

9 Testing

9.1 Scope

[TODO: What does testing check that verification doesn't, and the reverse? Where is the line between the two in this report?]

9.2 Testing Without the Robot

[TODO: How can the pipeline run with no robot attached? Which design choices made that possible, and when were they made?]

9.3 The Suite

[TODO: How large is the suite, what does it cover, and how long does it take to run? Which tests need the robot, and how are they selected?]

9.4 What Testing Proves

[TODO: Which requirements can software tests actually verify, and which can they not? Where did tests catch real bugs? Where did a passing suite hide a problem that only showed on the course?]

10 Diagnostics

10.1 Why Instrument the Pi

[TODO: What can go wrong on the Pi that the pipeline's own logs can't show? Why did that become worth building tools for?]

10.2 What Is Recorded

[TODO: Which recorders exist, and what question does each answer? Which ones actually changed a decision?]

Table 4: [TODO: caption]

Recorder	Question it answers
[TODO: rows]	

10.3 Observer Effect

[TODO: How do you know the recorder doesn't change the run it measures?]

10.4 What It Showed

[TODO: What did diagnostics reveal about frame rate, load, throttling or scheduling? Should the numbers go here, or only in Results?]

11 Verification

11.1 What Counts as Verified

[TODO: What has to be true before you call a requirement verified? Which hardware does it have to pass on, and do older results (IMX219) count?]

11.2 Routines

[TODO: What is a routine, and why run hardware checks this way instead of ad hoc? What does a request card fix in advance, and what does each run record?]

11.3 Checks by Requirement

[TODO: For each requirement: one check, with its pass criteria. Which requirements have no check defined yet?]

Table 5: **[TODO: caption]**

ID	Check and pass criteria
[TODO: rows]	

11.4 Whole-Robot Checks

[TODO: Which routines test the robot as a whole rather than a single requirement (figure-eight, course run, power profile ...)? What do they add?]

12 Results

12.1 Status by Requirement

[TODO: What grade does each requirement get, and on what evidence? Decide your grading scale first (e.g. Pass / Fail / Partial / Open) and define it in the caption.]

Table 6: **[TODO: caption, with the grading scale]**

ID	Grade	Evidence
[TODO: rows]		

12.2 Routine Results

[TODO: One row per routine run: routine, date, commit, trials, verdict, key number.]

12.3 Course Runs

[TODO: What happened when the robot ran the course? Early runs drifted right and failed at intersections. Has the stop-line fix resolved that?]

12.4 Measured Robot Response

[TODO: If the maneuver numbers live here rather than in Navigation, put them here.]

Part III
Assessment

13 Limitations

13.1 Requirements Not Met

[TODO: Which requirements fail, and what does each failure mean for how the robot drives?]

13.2 Requirements Not Yet Shown

[TODO: Which requirements have no hardware evidence either way? What is blocking each?]

13.3 Departures from the PRD

[TODO: Where does the robot's behavior differ from what the PRD asks?]

13.4 Limits of the Design

[TODO: What can this design never do well, however well it's tuned?]

14 Discussion

14.1 What Worked

[TODO: Which decisions would you make again, and what evidence says they were right?]

14.2 What Didn't

[TODO: Which decisions cost you, and when did you realize it?]

14.3 What I Would Do Differently

[TODO: With what you know now, what would you change in the design, the process, or how the team worked? How would you use AI-assisted development next time?]

14.4 What Remains

[TODO: What has to happen before Design Day, in priority order, and why that order?]

15 Conclusion

[TODO: If a reader only reads this section, what three things should they take away? What the pipeline is; how it measured against its requirements; what matters most next.]

References

G. Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000.

Brian Douglas. Autonomous navigation. MATLAB Tech Talks video series, MathWorks, 2020a. URL `https://www.mathworks.com/videos/series/autonomous-navigation.html`. Six parts. Accessed 2026-10-09.

Brian Douglas. Autonomous navigation, part 1: What is autonomous navigation? MATLAB Tech Talks, MathWorks, 2020b. URL `https://www.mathworks.com/videos/autonomous-navigation-part-1-what-is-autonomous-navigation-1592993748308.html`. Video, 11:29. Accessed 2026-10-09.

Mike Orduna, Ignacio Rubio, and Ana Carlos. Navilott Vision. `https://github.com/Simbyote/Navilott_Vision`, 2026.

Texas State University. AutoBots product requirements document. Technical report, Ingram School of Engineering, Texas State University, January 2026. Sponsored by Jeff Stevens and Mark Welker. Version 0.5.

Part IV
Supplements

A Hardware

[TODO: Parts list and pin map, for reference only. The reasons behind each part belong in Architecture.]

Table 7: **[TODO: caption]**

Part	Role	Interface
[TODO: rows]		

Table 8: **[TODO: caption]**

BCM	Header	Use
[TODO: rows]		

B Parameters

[TODO: Which tuning values does a reader need on record to reproduce your results? Group by phase.]

B.1 Capture

[TODO: Sensor mode, output size, frame rate.]

B.2 Perception

[TODO: ROIs, and the gates that matter.]

B.3 Estimation

[TODO: Filter, hold, vote and confidence settings.]

C The Estimation Packet

[TODO: Every field navigation receives per frame: name, type, meaning, units and sign convention.]

Table 9: **[TODO: caption]**

Field	Type	Meaning
[TODO: rows]		

D Repository

[TODO: Where the code is, how it is laid out, and the commands a reader needs to reproduce a result.]

D.1 Layout

[TODO: One line per directory under `vision_stack/src/`.]

D.2 Commands

[TODO: The few commands that matter: setup, every-boot session, run, test, routines.]